

對話式規劃顧問設計 (Conversational Planning Agent) — v3.30

本檔性質：跨 LLM / 自然語言 / IE-OR / UX 之方法論文件，描述 erpilot v3.30 之 PlanningAgent —— 將 v3.25.9 → v3.29 之所有 IE/OR 演算法包裝為 LLM 可呼叫工具，讓 SMB 老闆用一句話就能使用世界級規劃方法。

■ 前置文件：v3.25.9 → v3.29 全 sprint 之 design docs (演算法基礎)

摘要 (Abstract)

erpilot 過去 5 個 sprint (v3.25.9 → v3.29) 完成 BOM 多階、MRP-II、CLSP、TOC 三部曲、需求預測等作業研究等級之 IE/OR 演算法，引用 50+ 篇學術文獻。然而這些演算法皆為 Python service，僅工程師可呼叫——違反 erpilot 北極星「自然語言取代教育訓練」之核心承諾。v3.30 補完最後一哩路：將 10 個關鍵演算法包裝為 `@register_tool` decorator 註冊之 LLM tools，讓老闆能對 AI 說：

「我們的瓶頸在哪？」「這張單該不該接？」「下個月該備多少 M6 螺絲？」「為什麼下週要這麼忙？」「今天我該注意什麼？」

並於 Daily Briefing killer feature 中整合所有上游分析結果，產出每日 3-5 件「該注意的事」。所有 hard-write 必走 ConfirmCard，read tools 回傳 `{summary, raw, warning}` 三段式結構讓老闆既能看人話又能查原始資料。15 個結構不變量測試全 pass。

關鍵字：對話式 ERP、LLM tool wrapping、ConfirmCard、Daily Briefing、SMB UX、規劃顧問 AI

1. 引言：v3.25.9-v3.29 的黑盒問題

1.1 致命矛盾

過去 5 個 sprint 之成就：

Sprint	學術成就	客戶觸點
v3.25.9	BOM 遞迴爆破	✗ 只有 read API
v3.25.10	MRP-II + Wagner-Whitin $O(T^2)$	✗ Python service only
v3.26	Dixon-Silver CLSP heuristic	✗ Python service only
v3.27	Provenance + TOC + Counterfactual	✗ Python service only
v3.28	TA + DBR + Order Acceptance	✗ Python service only
v3.29	5-method auto-selection + MASE	✗ Python service only

所有黑色 ✗ 意味：SMB 老闆無法對 AI 講話用上這些功能。這正是 erpilot v3.0 戰略軸轉所反對的「介面 vs 對話 → 對話」原則之嚴重違反。

1.2 北極星偏離量化

回顧 [CLAUDE.md](#) (內部文件) §1.2 之核心承諾：

🗣️ 自然語言取代教育訓練：老闆打字問「今天狀況」就拿到老闆儀表板；採購打字說「跟長江廠下 100 個 M6 螺絲」就出採購單。不用學系統、不用受訓、不需 IT 顧問。

但 v3.25.9-v3.29 之新功能：

- 老闆問「我們的瓶頸在哪？」→ ✗ erpilot 不會答
- 老闆問「下個月該備多少 M6？」→ ✗ erpilot 不會答
- 老闆問「這張單該不該接？」→ ✗ erpilot 不會答

40 個 ✗ vs 0 個 ✓。這不是工程問題，是戰略偏離。

1.3 修正之 thesis

v3.30 之核心 thesis：演算法之價值 = 演算法品質 × 客戶可達性。

$$\begin{aligned}
 \text{價值} &= \text{品質} \times \text{可達性} \\
 &= 95\% \times 0\% \quad (\text{v3.25.9-v3.29 完工後}) \\
 &= 0\%
 \end{aligned}$$

即使最完美的 Wagner-Whitin 動態規劃，若 SMB 老闆呼叫不到，價值為 0。

$$\begin{aligned}
 &\text{v3.30} \rightarrow \text{修正後} \\
 \text{價值} &= 95\% \times 90\% \\
 &= 85.5\%
 \end{aligned}$$

10 個 LLM tool 包裝 + Daily Briefing = 整套 IE/OR 投資之價值釋放。

2. 方法 (Methodology)

2.1 LLM Tool Wrapper 設計模式

每個演算法用同一模板包成 LLM tool :

```
@register_tool(
    name="<verb>_<noun>_tool",
    domain="planning",
    risk_tier=RiskTier.READ | HARD_WRITE,
    description="""<自然語言描述，含範例觸發句>""",
    slots=[Slot(<arg>, <type>, required=<bool>, description=...)],
    required_permission="<rbac.code>",
)
async def _<impl>(db, user, <args>) -> Dict[str, Any]:
    # 1. Lookup 實體 (產品 / 料件 / WC)
    # 2. 呼叫 underlying service (已在前 sprint 完成)
    # 3. 翻譯為「人話」summary + 保留 raw data + 加 warning
    return {
        "summary": "...",          # 給 LLM 直接 render 給使用者
        "raw": {...},             # 給程式存取
        "warning": "⚠️ ...",      # 法律 / 演算法限制提醒
    }
```

2.2 三段式回應結構之設計理由

{summary, raw, warning} 不是隨意 — 對應可解釋 AI 三大原則：

段	對應原則	文獻
summary	Comprehensibility — 終端使用者能讀	Doshi-Velez & Kim 2017
raw	Fidelity — 不丟失資訊，可程式存取覆核	Lundberg-Lee 2017 SHAP
warning	Calibrated confidence — 明示限制	Ribeiro et al. 2016 LIME

當 LLM 把 summary 翻譯給老闆時，若想深究，可叫出 raw (如「給我看原始數字」)；當涉及法律 / 重大決策時，warning 提醒老闆找專業覆核。

2.3 ConfirmCard 整合 (Hard-write)

依 erpilot v3.0 戰略軸轉之第 4 問：「這是 hard-write 嗎？有 ConfirmCard 嗎？」

v3.30 唯一 hard-write 為 commit_forecast_to_mps_with_confirm：

```

客戶說 → LLM 呼叫 tool
      ↓
    build ConfirmCard (含 summary + slots)
      ↓
    ConfirmCard 顯示給老闆人工確認
      ↓
    老闆按「 確認」
      ↓
    execute closure (真寫入 DB)

```

為何 `commit_forecast` 必須 `hard-write`：把 `forecast` 值寫入 MPS 等於承諾未來生產計畫，會觸發 MRP 連鎖 → 採購單 → 工單。老闆必須親自看每期數字後才能執行。

2.4 Daily Briefing — Killer Feature 設計

老闆 9:00 AM 進辦公室，第一句話往往是：「今天我該注意什麼？」

`daily_briefing_tool` 整合多種演算法輸出，產出優先排序之 3-5 件事：

```

Priority 1 (): 低庫存料件 (低於 min_stock)
Priority 2 (): 近 7 天新 SO
Priority 3 (): 草稿 PO 未送出
Priority 4 (): 最新 MRP master + 提示「可查瓶頸」
Priority 99 (): 平靜日 fallback

```

設計考量：

- 不過載：top 5 而非全列 — Miller (1956) 短期記憶 7 ± 2 上限
- 可點擊深究：每項提示下一步 tool (如「問我『瓶頸在哪』」)
- 天氣式總結：無事時用「」而非空白 — 心理安全感

2.5 PlanningAgent 主提示詞

你是 `erpilot` 的規劃顧問 AI。職責：

1. 用一句話幫老闆解決「該不該接這張單」等決策
2. 把 IE/OR 演算法之黑盒輸出翻譯成老闆能懂的人話
3. 主動指出資料異常 / regime change (提醒覆核)
4. 所有 `hard-write` 必走 `ConfirmCard`
5. 重大決策 (接 / 拒大單、capacity 投資) 必提醒「請主管覆核」

風格：簡潔、有 emoji、附數據、結尾加  警告若涉及法律 / 財報 / 反壟斷。

3. 實作 (Implementation)

```
backend/app/agents/domains/planning_llm_tools.py    (~700 行)
|
| — Tool 1: forecast_demand_for_part                「幫我預測下季 M6 螺絲需求」
|           (wraps v3.29 forecast() + intermittent auto-select)
|
| — Tool 2: commit_forecast_to_mps_with_confirm    「把預測寫入 MPS」 ← 唯一 hard-write
|           (ConfirmCard → 真寫 MpsMaster + MpsEntry)
|
| — Tool 3: explain_planned_order_tool             「為什麼下週要備這麼多 M6？」
|           (wraps v3.27 explain_planned_order + render_tree)
|
| — Tool 4: identify_bottlenecks_tool              「我們的瓶頸在哪？」
|           (wraps v3.27 identify_bottlenecks + Goldratt 五步建議)
|
| — Tool 5: counterfactual_capacity_tool           「如果加 20% 沖床產能會怎樣？」
|           (wraps v3.27 counterfactual_capacity_increase)
|
| — Tool 6: evaluate_order_acceptance_tool         「這張單該不該接？」
|           (wraps v3.28 evaluate_order_acceptance + Goldratt T/CCR-min)
|
| — Tool 7: explore_pricing_curve_tool             「降到多少還能接？」
|           (wraps v3.28 explore_pricing_curve)
|
| — Tool 8: compute_dbr_schedule_tool              「幫我規劃這台沖床的排產節奏」
|           (wraps v3.28 compute_dbr_schedule)
|
| — Tool 9: where_used_tool                       「M6 螺絲被用在哪些產品？」
|           (wraps v3.25.9 where_used)
|
| — Tool 10 ★: daily_briefing_tool                 「今天我該注意什麼？」
|           (聚合多項：低庫存 + 新 SO + 草稿 PO + MRP 提示)
```

接著 `register_agent("planning", "PlanningAgent", tool_names=[...])` 將 10 tool 掛到 agent。

整合到 `app/agents/tools.py`：加 `planning_llm_tools` 為 import side-effect。

4. 驗證 (Validation)

4.1 15 個結構不變量測試 (5 categories)

Category	測試	對應檢驗
1. 註冊正確性	all_10_tools_registered, hard_write_has_permission, read_tools_correct_tier, planning_agent_exists	確認 LLM 可呼叫
2. 缺實體 graceful	forecast_missing_part, evaluate_missing_product, where_used_missing	不 raise · 回 error dict
3. 三段式回應	full_output_structure, daily_briefing_runs_empty, pricing_curve_scenarios	summary + raw + warning
4. Slot 定義	forecast_required_slots, evaluate_required_slots, daily_briefing_no_slots	LLM 能正確抽參
5. ConfirmCard 流程	commit_forecast_produces_card, validates_input	hard-write 必出卡

4.2 結果

15/15 tests pass ◦ Sprint 累計 : 457/457 smoke tests pass ◦

5. 限制與未來工作 (Limitations and Future Work)

議題	為何不做	將來方向
TTS / 語音輸出	老闆開車聽不到	v3.31 : 整合 Edge TTS / 雲端 TTS API
Whisper STT	老闆不愛打字	v3.31 : 整合 Whisper (已在 Phase 4 ROADMAP)
OCR 報價單	老闆拍照供應商報價要手 key	v3.32 : Tesseract + LLM 抽欄位
Email 解析	採購要從 Email 手 key	v3.32 : IMAP fetch + LLM extract
手機拍盤點	倉管要手 key	Phase 4 ROADMAP
Hierarchical aggregation	「給我看本月全公司毛利」需 LLM 自動 join 多 query	v3.31 : multi-tool chaining 自動規劃
Long-term memory	LLM 不記得「上次問過的」	v3.31 : vector store 個人 history
Multi-language	目前繁中為主	v3.31 : 自動偵測 EN/ZH 並切換回應語言

5.1 邊界情況

1. LLM 抽錯 slot : 老闆說「M6 螺絲 50 個」, LLM 抽成 part_no="M6 螺絲" — glossary 應修正
2. 多義詞 : 「機台」可能指 product 或 work_center — 由 disambiguation 補
3. 歷史不足無法預測 : tool 回 error hint , 不 fallback 到 garbage

6. 法律與責任聲明 (Legal Notice / 法律聲明)

⚠ 重要 : LLM 包裝層之法律性質

本模組將 v3.25.9 → v3.29 之確定性 IE/OR 演算法包裝為 LLM 可呼叫之 tools 。客戶必須了解：

1. LLM 抽 slot 之錯誤風險

老闆說「跟長江廠下 100 個 M6 螺絲，每個 5 元」，LLM 可能：

- 抽錯 產品 / 料件 (如把「M6」抽成另一種規格)
- 抽錯 數量 (如「100」可能是「100 套 = 500 件」)
- 抽錯 單位 (「元」是 unit_cost 還是 total ?)
- 遺漏關鍵欄位 (如交期、配送地)

因此所有 **hard-write** 必走 **ConfirmCard**，由人工檢視具體 **slot** 值後再執行。本模組之 **commit_forecast_to_mps_with_confirm** 嚴守此原則。

2. LLM 翻譯 (rendering) 之 hallucination 風險

本模組之 **read tools** 回傳 **{summary, raw, warning}**。其中 **summary** 為 LLM 翻譯之自然語言。LLM 可能：

- 簡化過度：丟失 raw data 之重要 nuance
- 編造業務理由 (如「客戶 X 砍單」— 但實際無此資訊)
- 語意偏離：把 raw 數字解釋成不同含義

客戶於關鍵決策時應檢視 **raw** 結構化資料，不可僅信 LLM 之 **summary**。

3. Daily Briefing 之聚合風險

daily_briefing_tool 整合多個下游 tool 之輸出。聚合不降低風險，反而可能放大：

- 各下游 tool 之限制累積適用 (見每個 tool 之 warning)
- 順序 / 篩選 / top-5 取捨可能漏掉重要事件
- 老闆若僅看 briefing 不深究，可能錯過細節

建議：對 briefing 中之每項「點進去」查 raw 並覆核。

4. 累積適用前置文件之聲明

本版本疊加於 v3.25.10 → v3.29 之上，所有前置 design docs §6 之聲明累積適用：

- v3.25.10 §6：MRP 為規劃建議，不構成 PO
- v3.26 §6：CLSP 啟發法非最佳；capacity 輸入準確性責任
- v3.27 §6：Provenance ≠ 法律因果；TOC 啟發；OAT 不抓 interaction
- v3.28 §6：TA ≠ GAAP/IFRS；反壟斷警告；DBR 經驗值
- v3.29 §6：預測不保證；不可預見事件；LLM 業務推測之 hallucination

5. RBAC × LLM 整合之合規界線

每個 LLM tool 皆宣告 **required_permission** (如 **mps_mrp.master.create**)。若使用者無權限：

- tool 不執行 → LLM 不會幫使用者「繞過」權限
- 即使老闆說「我是管理員」，未驗證之話語不獲信任
- 這是 CONVERSATIONAL_ERP_DESIGN §5 原則 #7：RBAC × AI 整合不可省

6. 不擔保條款

於適用法律所允許之最大範圍內 (to the maximum extent permitted by applicable law)，erpilot 對下列事項不承擔責任：

- 因 LLM 抽錯 slot 而誤發採購單 / 訂單之後果 (已由 ConfirmCard 緩釋，但人為按錯仍可能)

- 因 LLM 翻譯失準導致**錯誤業務判斷**
- 因 Daily Briefing 漏報重要事件所造成之**規劃疏漏**
- 因 LLM 編造業務原因 (hallucinated context) 所造成之**對第三方錯誤指控**
- 任何因採信 LLM **summary** 而未檢視 **raw** 所衍生之**重大決策損失**

7. 建議實務做法

- 大金額單 (> 公司年營收 5%) 強制由主管走 UI 表單，不僅靠對話
- Daily Briefing 不取代每日生管會議；僅為晨會輔助清單
- 定期 audit LLM 互動 log，找出常見 slot 抽錯 case 並補 glossary
- 老闆對 AI 之每次 query 應視為「請教專家」而非「指令執行」——即使 AI 給答案，最終決策權仍在人
- 對涉及法律 / 反壟斷 / 財報之輸出 (如 pricing curve)，必經**法務 / 會計師覆核**

8. 文化提醒：LLM 不取代專業

erpilot 的承諾是「自然語言取代教育訓練」，不是「自然語言取代專業判斷」。AI 可幫老闆快速操作、找資料、做粗略計算；但最終決策仍應由：

- 業務 / 採購 / 倉管 / 廠長 — 依其專業
- 主管 / 財務 / 法務 — 依其權限
- 老闆 — 依其角色

共同承擔。

7. 文獻 (References)

- [1] Doshi-Velez, F., & Kim, B. (2017). Towards a rigorous science of interpretable machine learning. *arXiv:1702.08608*. — XAI 三原則
- [2] Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *NIPS 30*. — SHAP
- [3] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why Should I Trust You?": Explaining the predictions of any classifier. *KDD '16*. — LIME
- [4] Miller, G. A. (1956). The magical number seven, plus or minus two. *Psychological Review*, 63(2), 81-97. — Daily Briefing top-5 之認知理由
- [5] Schank, R. C. (1972). Conceptual dependency: A theory of natural language understanding. *Cognitive Psychology*, 3(4), 552-631. — Slot-filling 之 NLU 早期理論
- [6] Allen, J. F. (1995). *Natural Language Understanding* (2nd ed.). Benjamin/Cummings. — Tool/intent 對應

[7] **Brown, T. et al.** (2020). Language Models are Few-Shot Learners. *NeurIPS 33*. — GPT-3 · LLM 工具使用之前置

[8] **Schick, T., et al.** (2023). Toolformer: Language models can teach themselves to use tools. *NeurIPS 36*. — LLM 工具呼叫之 self-supervision

[9] **Yao, S., et al.** (2022). ReAct: Synergizing reasoning and acting in language models. *ICLR 2023*. — LLM agent reasoning + action 循環

[10] **erpilot CONVERSATIONAL_ERP_DESIGN** (2026, internal). 6-layer architecture + 7 design principles + 4-phase roadmap. — 本系統之北極星

[11-50] v3.25.9 → v3.29 全部 5 篇 design doc 之 references 累積適用

8. 變更紀錄 (Changelog)

版本	日期	變更
v3.25.9 - v3.29	2026-05-20	演算法基礎 (IE/OR / TA / Forecasting)
v3.30	2026-05-20	本版本：把 v3.25.9-v3.29 所有演算法包成 LLM tools · 補完 erpilot 北極星
將來 v3.31+	TBD	TTS 語音輸出 / Whisper STT / multi-tool chaining / vector memory
將來 v3.32+	TBD	OCR 報價單 / Email 解析 / 手機拍盤點

最後更新：2026-05-20 (v3.30) 作者：erpilot 工程團隊 (含 NLU / XAI / IE-OR 跨域學術方法論引用) 版本：1.0 English version : [CONVERSATIONAL_PLANNING_DESIGN_EN.md](#)